A Multi-Stage Approach to Plagiarism Detection On The arXiv Dataset

Big Data Mining Project (52017)

Yaron Otmazgin & Yael Levy

Abstract

Academic plagiarism detection in large-scale document collections presents significant computational challenges due to the quadratic complexity of pairwise comparisons. This paper presents a multi-stage pipeline designed to detect plagiarism among approximately one million arXiv LaTeX papers while maintaining computational efficiency. Our approach combines semantic clustering using HDBSCAN, efficient textual overlap detection through Bloom filters, and deep semantic similarity analysis using a Siamese BERT architecture. The system first groups semantically similar papers through abstract-based clustering, then applies Bloom filters for fast n-gram overlap detection, and finally employs a trained Siamese BERT model for nuanced plagiarism detection including paraphrasing and conceptual duplication. The pipeline addresses the dual challenges of computational scalability and detection accuracy in large academic corpora.

1 Introduction

The exponential growth of academic publications has intensified the need for automated plagiarism detection systems capable of processing large-scale document collections. Traditional pairwise comparison approaches become computationally prohibitive when applied to repositories containing millions of documents, as they require $O(n^2)$ comparisons. Meanwhile, simple text-matching techniques fail to detect sophisticated forms of plagiarism involving paraphrasing, translation, or structural modifications.

This work addresses these challenges by proposing a multi-stage plagiarism detection pipeline that combines computational efficiency with detection sophistication. Our system processes approximately one million LaTeX papers from the arXiv repository through three sequential stages: semantic clustering to reduce search space, Bloom filter-based n-gram analysis for efficient textual overlap detection, and Siamese BERT-based semantic analysis for comprehensive plagiarism assessment.

The key contributions of this work include: (1) a scalable multi-stage architecture that reduces computational complexity while maintaining detection accuracy, (2) an adaptation of Bloom filters for academic plagiarism detection with configurable sensitivity parameters, and (3) a Siamese BERT implementation trained on the PAN Plagiarism Dataset[1] and optimized for academic text similarity assessment.

2 Naive Computational Complexity

The multi-stage filtering approach is essential to ensure computational feasibility when processing large-scale document collections. Without semantic clustering and Bloom filter preprocessing, a naive approach would require exhaustive pairwise comparison of all text segments across the corpus. Specifically, for plagiarism detection, each paragraph or chunk from every paper would need to be compared against all paragraphs in all other papers.

For a corpus of $N = 10^6$ papers, each containing $P = 50$ paragraphs, the total number of paragraph-level comparisons in a naive approach is:

$$C_{\text{naive}} = N \cdot P \cdot (N-1) \cdot P \approx 1.25 \times 10^{15} \qquad (1)$$

Assuming each BERT similarity computation takes $t = 0.05$ seconds on a single GPU, the total computation time would be approximately 2 million years. Even leveraging batching on a state-of-the-art NVIDIA A100 GPU reduces the effective per-comparison time by roughly three orders of magnitude, lowering the runtime to $\sim 1,936$ years on a single GPU, or ~ 242 years using an 8-GPU cluster—infeasible.

In contrast, our pipeline reduces this complexity by over six orders of magnitude. Semantic clustering partitions papers into topically coherent clusters, so that comparisons are performed **only within clusters**, rather than across the entire corpus. Bloom filters further eliminate approximately 90% of unlikely candidate paragraph pairs within each cluster through fast n-gram overlap detection. These stages together reduce the number of BERT comparisons to fewer than 5×10^8. Assuming sequential computation on standard hardware (without aggressive GPU op-

timization), the total runtime is then approximately:

$$T_{\text{filtered}} = 5 \times 10^8 \cdot 0.05 \text{ s} = 2.5 \times 10^7 \text{ s} \approx 289 \text{ days}.$$

With GPU batching (batch size $B = 512$), the number of batches is

$$\text{num\_batches} = \frac{5 \times 10^8}{512} \approx 976{,}562,$$

yielding a total runtime of

$$T_{\text{batches}} = 976{,}562 \cdot 0.05 \text{ s} \approx 48{,}828 \text{ s} \approx 13.6 \text{ hours},$$

showing that modern GPUs can reduce the runtime from hundreds of days to **less than a day**. Thus, the proposed multi-stage pipeline makes large-scale plagiarism detection computationally feasible compared to the naive approach.

3 Related Work

Plagiarism detection research has evolved from simple string-matching algorithms to sophisticated semantic analysis techniques. Early approaches relied on exact text matching and statistical fingerprinting methods. More recent work has incorporated machine learning techniques, with neural network-based approaches showing particular promise for detecting semantic plagiarism.

The PAN (Plagiarism Analysis, Authorship Identification, and Near-Duplicate Detection) workshops have established benchmark datasets and evaluation metrics for plagiarism detection research. The PAN corpus includes manually annotated plagiarism instances with various obfuscation levels, providing ground truth for supervised learning approaches.

Large-scale document processing has benefited from advances in approximate matching algorithms and probabilistic data structures. Bloom filters, originally designed for database applications, have found applications in text processing due to their space efficiency and fast membership testing capabilities.

4.1 System Architecture

The plagiarism detection pipeline is composed of sequential stages aimed at balancing computational efficiency and detection accuracy. The system follows a funnel-like architecture, gradually narrowing down the set of candidates while applying more precise and targeted analysis at each stage (Figure 1).

This pipeline processes approximately 1 million arXiv papers through an efficient filtering cascade.

Stage 1 begins with the raw LaTeX corpus. **Stage 2** uses HDBSCAN clustering on paper abstracts to group topically similar documents, reducing unnecessary comparisons by 90%, resulting in 308 clusters. **Stage 3** applies Bloom filters within each cluster to perform efficient n-gram analysis, identifying paper pairs with significant textual overlap while filtering out same-author papers (self-plagiarism). This stage identifies approximately 4,000 candidate pairs. **Stage 4** performs computationally expensive Siamese BERT semantic analysis on the candidates from the Bloom filter stage, using trained model weights from the PAN plagiarism dataset. This stage produces detailed similarity scores and identifies specific text chunks exceeding the plagiarism threshold. **Stage 5** ranks all results by similarity score and priority level. The results include the specific text passages that triggered plagiarism detection, enabling efficient human review and validation.

4.2 Training Data And Ground Truth

The Siamese BERT model (model's architecture explained in chapter 4.5 and Figure 8) is trained using the PAN Plagiarism Dataset, which provides manually annotated instances of plagiarism. This dataset encompasses a range of obfuscation techniques, including paraphrasing, translation, and structural modifications, allowing the model to learn a wide variety of plagiarism patterns.

In the implementation of the training process (`plagiarism_detector.py`), plagiarism instances are filtered to include both low and high obfuscation types. Initially, the model was trained using only low-obfuscation instances; however, during inference on real-world paper instances, the similarity scores between closely related passages were often low. This caused the model to focus narrowly on instances exhibiting obvious plagiarism (e.g., verbatim copying), limiting its ability to detect more subtle forms such as paraphrasing. To address this, high-obfuscation

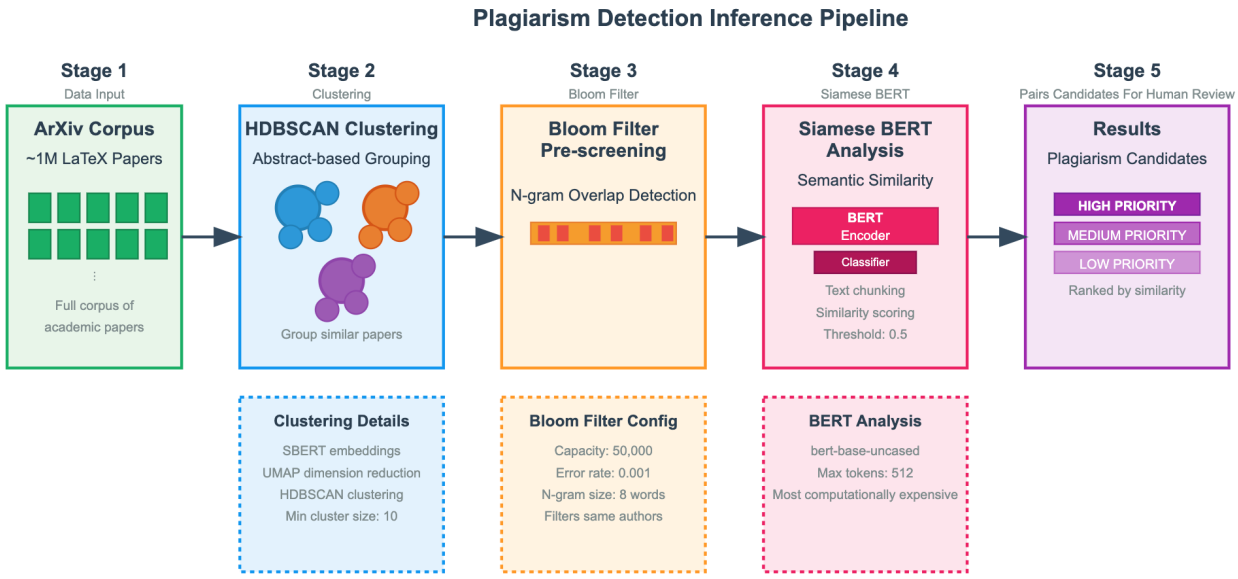

Figure 1: *Plagiarism Detection Pipeline*

instances were included in training.

Before optimization, each text pair in the dataset was tokenized on-the-fly for every epoch, resulting in a complexity of $O(N \cdot E \cdot L)$, where N is the number of pairs, E the number of epochs, and L the average token length per passage. For example, with 20,000 pairs, 5 epochs, and passages of 512 tokens, this amounts to roughly 102.4 million tokenization operations. After pre-tokenizing all passages once during dataset initialization, the complexity reduces to $O(N \cdot L)$, requiring only ~ 20.5 million tokenization operations. This eliminates repeated work across epochs and reduces the tokenization cost by a factor of ~ 5, while data retrieval during training becomes an $O(1)$ operation per sample.

The number of plagiarized instances (both low and high obfuscation) matches the number of non-plagiarized instances, ensuring a balanced distribution (Figure 2).

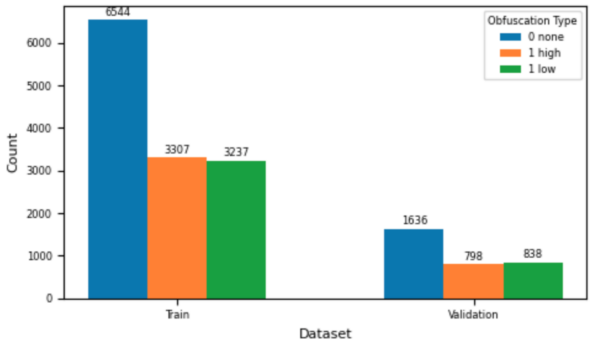

Figure 2: *Distribution Of Obfuscations*

At first, negative pairs were generated by randomly pairing unrelated passages. However, when evaluating the model on plagiarism instances with differing lengths, performance was poor. The model had essentially learned to distinguish positive and negative pairs based on length differences rather than semantic similarity (e.g the model relied on the length difference between instances as an indication for non-plagiarism) so even a nearly exact copy missing a few

sentences would score low (Figure 3 verifies it).

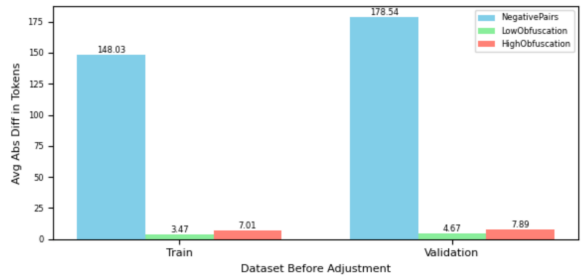

Figure 3: *Before Negative Pairs Adjustment*

We also observe that high-level obfuscations tend to produce larger absolute length differences, suggesting that such cases represent a more sophisticated form of plagiarism. To address this, we modified the negative sampling process to match the length distribution of positive pairs. For each negative pair, a target length difference was sampled from the positive pairs, and candidate passages were selected such that their length difference approximated this target (Figure 4). This approach prevented the model from relying on length cues and improved its ability to detect plagiarism across passages of varying lengths.

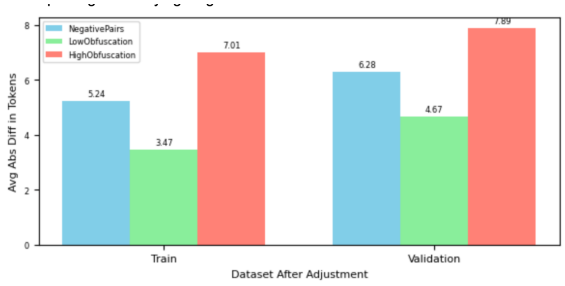

Figure 4: *After Negative Pairs Adjustment*

We observed that applying dropout to the embeddings can harm plagiarism detection. Dropout randomly zeros parts of the embedding, which can remove important information about key words or phrases (e.g., specific technical terms that indicate copying), and it also changes the embedding's direction, affecting similarity calculations. To address this, we removed dropout from the embeddings but retained it in the classifier layers. This preserves critical signals for detecting plagiarism while still providing regularization in the later layers.

4.3 Semantic Clustering

The clustering implementation in `tex_clustering.py` demonstrates the semantic grouping approach. Papers are processed by extracting abstracts from LaTeX files using comprehensive pattern matching to handle various abstract formatting conventions. The extracted abstracts are then converted into embeddings using the `all-MiniLM-L6-v2` sentence transformer model.

Dimensionality reduction is performed with UMAP (Uniform Manifold Approximation and Projection) using 5 components, followed by HDBSCAN clustering with a minimum cluster size of 10. This configuration balances cluster cohesion with computational efficiency.

To interpret the clusters, we identified the top three recurring words per cluster after removing standard and domain-specific stopwords. Figure 5 reveals meaningful groupings, for instance, clusters about galaxies, emissions, and stars appear near clusters discussing neutrinos, mass, and energy, while clusters on models, learning, and language are close to those on deep neural learning (Figure 5). This demonstrates that the semantic clustering effectively captures topic relationships across the corpus.

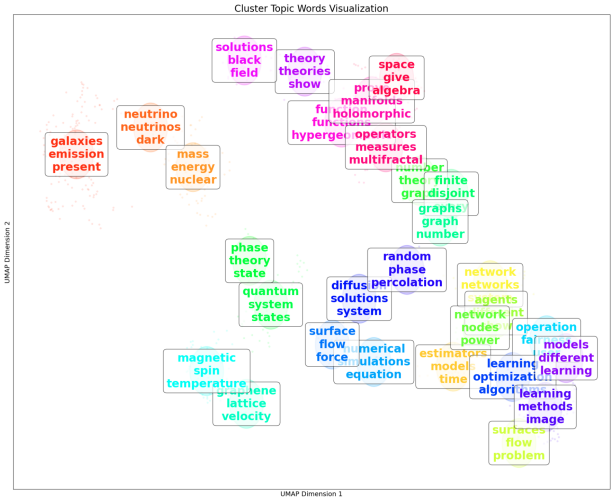

Figure 5: *Clusters After Dimensional Reduction*

4.4 Bloom Filter Implementation

Our Bloom filter uses 5 hash functions to convert text phrases (n-grams) into positions in a large bit array, setting those positions to 1. When comparing two documents, identical phrases set the same bit positions in both filters, and the system counts the number of positions with 1s in both. A high overlap suggests that the documents share many phrases, although different phrases may hash to the same positions, producing false positives. Importantly, the Bloom filter does not produce false negatives—it will never fail to flag a recurring passage, though it may occasionally flag a non-recurring passage.

In our implementation (`bloom_pipeline.py`), the filter is configured with a capacity of 50,000 items, an error rate of 0.001, and 8-gram analysis. Text preprocessing removes LaTeX formatting, mathematical expressions, and non-textual environments while preserving semantic content. Author verification reduces false positives by filtering pairs from the same authors.

During preprocessing, we observed Bloom filter saturation, with too many bits set to 1, increasing the false-positive rate. The probability of a false positive is approximately:

$$P_{\text{fp}} \approx (1 - e^{-kn/m})^k \tag{2}$$

where m is the filter capacity in bits, k is the number of hash functions, and n is the number of inserted elements. To mitigate saturation, we increased m, improving accuracy at the cost of additional computational resources.

Out of the 308 clusters, the Bloom filter found approximately 890 pairs of papers with actual overlapping n-grams (after filtering out false positives pairs that initially had some mutual bits activated but turned out to be different instances) (Figure 6).

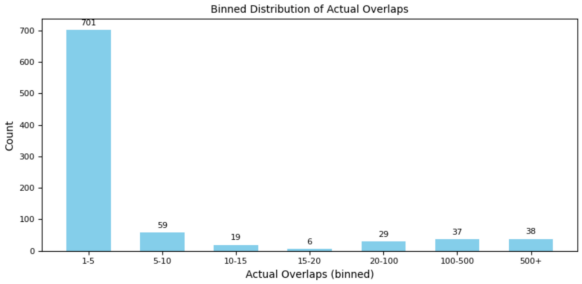

Figure 6: *Distribution Of n-gram Overlappings*

The binned distribution illustrates the extent of n-gram overlaps across all detected pairs, which may signal varying degrees of textual similarity. With 8-grams being highly specific sequences, the 701 pairs (78.7%) showing 1–5 overlaps represent potential plagiarism cases, while 78 pairs (8.7%) with 5–15 overlaps indicate stronger similarity signals. The remaining 110 pairs (12.3%) with 15+ overlaps suggest a high likelihood of plagiarism, ranging from substantial text reuse (20-100 overlaps) to extensive copying (100+ overlaps) or same-author papers that escaped filtering due to author name mismatches (for instance, in one paper the author name is *Josh Levi* and in the other it is *Joshua L.* or *J.L.*).

It is worth noting that many instances may involve general academic phrases (e.g "in this paper, we show that the results"...) that would be filtered out in the next step of our plagiarism detection pipeline using a Siamese BERT model, which analyzes the semantic similarity beyond exact n-gram matches.

An illustrative example emerges from cluster 118 (papers about theoretical physics), where the Bloom filter identified two papers by distinct authors exhibiting 496 actual overlapping n-grams. The first paper, *"QBism, FAPP and the Quantum Omelette"* (arXiv:1608.00548v1), was published by Christian de Ronde on August 1, 2016. Subsequently, on August 20, 2016, Ulrich Mohrhoff published *"QBism, Bohr, and the Quantum Omelette Tossed by de Ronde"* (arXiv:1608.05780v1), which explicitly aims to critically examine de Ronde's theoretical claims regarding QBism. The substantial textual overlap results from Mohrhoff's extensive quotation of de Ronde's work as part of his analytical critique (Figure 7).

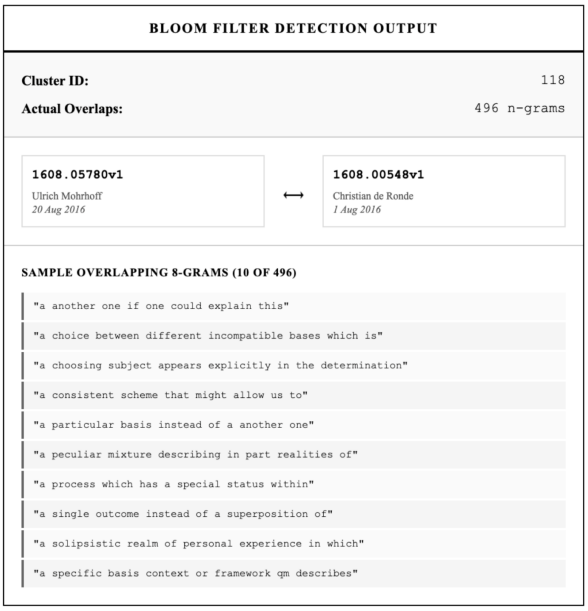

Figure 7: *Bloom Filter Overlapping Example*

This instance demonstrates the Bloom filter's operational efficacy in identifying genuine textual similarities while highlighting the necessity of human oversight in distinguishing between illegitimate copying and legitimate scholarly citation practices. Even with subsequent Siamese BERT analysis, this pair would likely maintain high similarity scores due to the extensive quotations. Without human review, an automated system would erroneously classify Mohrhoff's work as plagiarizing de Ronde's paper, when in fact it represents legitimate scholarly discourse involving direct quotation for critical analysis.

4.5 Siamese BERT

In order to detect more nuanced instances of plagiarism, and not only overlapping instances of n-grams, we trained a Siamese BERT model to identify plagiarism instances based on the PAN dataset. The model is designed to detect semantic similarity between pairs of text passages for plagiarism detection. It leverages a shared BERT encoder (bert-base-uncased) to generate contextual embeddings for each input, ensuring that both texts are processed with identical parameters. The model architecture (Figure 8) is tailored to capture both individual and relational features by combining the embeddings and their absolute difference. To address initial training instabilities, the classifier employs LeakyReLU activations and dropout regularization, which improved convergence and performance (the model initially struggled due to vanishing gradients with ReLU, but switching to LeakyReLU resolved the issue and improved performance). Training is performed using binary cross-entropy loss, AdamW optimization, and a linear learning rate scheduler.

Based on the training results on the PAN dataset (Figure 9), the model demonstrates decent convergence, with the training loss decreasing steadily from 0.5 to approximately 0.07 over 5 epochs, while the validation loss stabilizes around 0.21–0.22 after epoch 2, indicating good generalization without significant overfitting. The validation accuracy reaches 91.3% by the final epoch, showing that the Siamese BERT model has successfully learned to distinguish between plagiarized and non-plagiarized text pairs with high reliability. Notably, since our training dataset included both low and high obfuscation types, this high accuracy indicates that the model effectively learned to detect plagiarism instances with significant rephrasing and paraphrasing, not just direct text copying.

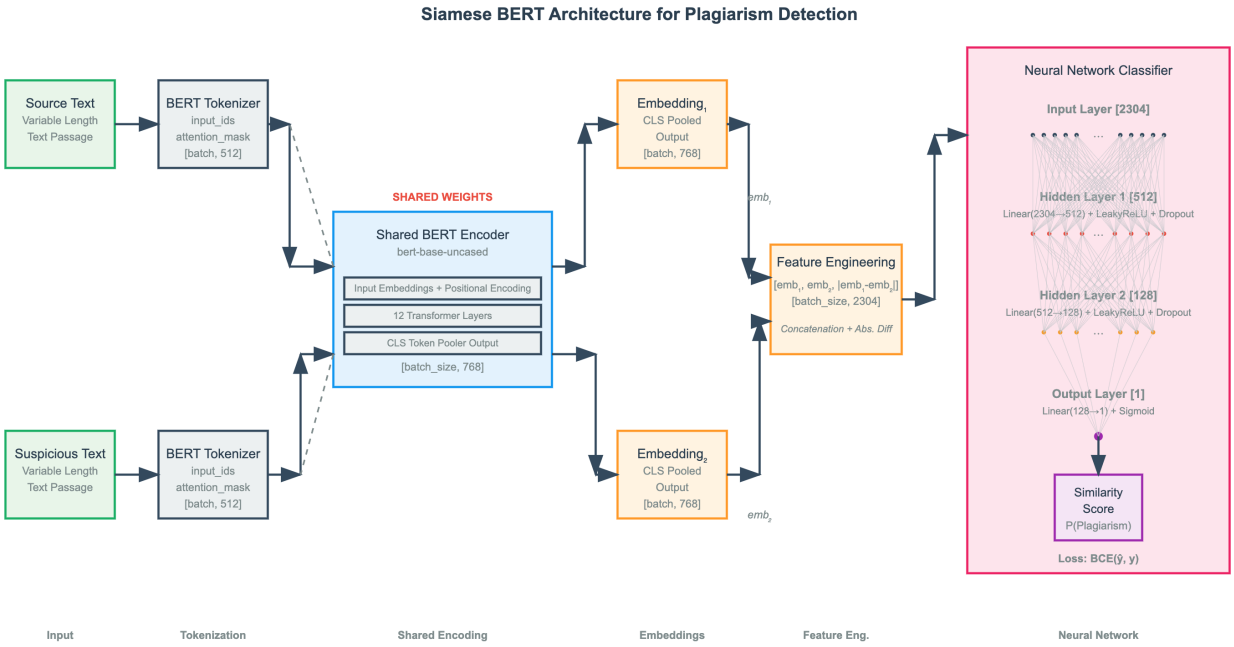

Figure 8: *Each input text is tokenized and converted into a sequence of up to 512 tokens, accompanied by an attention mask. Both sequences are processed by a shared BERT encoder (12 transformer layers, 768-dimensional output per input). The resulting embeddings (2 × 768) are concatenated with their absolute difference, forming a 2304-dimensional feature vector. This vector is passed through a feedforward neural network with layer sizes 2304→512→128→1, using LeakyReLU activations and dropout (0.3) between layers. During training, the model outputs a probability score via a sigmoid activation and is optimized with binary cross-entropy loss.*

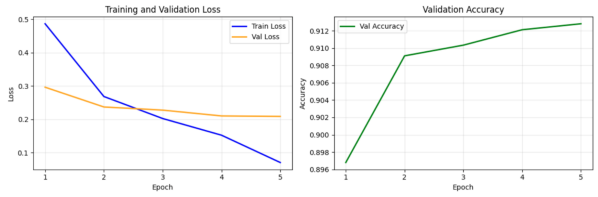

Figure 9: *Training results based on optimized parameters after using hyperparameter tuning: lr of 2e-5 with scheduler, batch size of 8, 0.5 threshold (for validation only), 5 epochs.*

The trained model weights are then applied to candidate pairs identified by the Bloom filter stage, processing only those pairs that exceed a predefined threshold of actual overlapping n-grams.

4.6 Defining Passages & Applying Weights

To apply the trained model to candidate paper pairs, we must first address the challenge of defining appropriate text passages. While paragraphs represent natural semantic units, our Siamese BERT model has a 512-token input limit, causing longer paragraphs to exceed this constraint. During training, passages exceeding the token limit were filtered out to maintain dataset quality. Initially, we applied the same filtering approach during inference, simply discarding oversized passages. However, this strategy proved problematic, as it eliminated potentially important content and reduced the number of valid comparisons, potentially causing the model to miss

plagiarism instances. To resolve this issue, we implemented an adaptive chunking strategy. Rather than truncating passages at the 512-token boundary, we employ a hierarchical splitting approach that first divides text at natural paragraph boundaries. When paragraphs exceed the token limit, they are further segmented at sentence boundaries to preserve semantic coherence. For paper comparison, the system generates all possible chunk pairs and processes them in batches to optimize GPU utilization.

5 Scalability Considerations

The whole pipeline includes built-in safeguards to manage computational resources and prevent system overload. Documents longer than 50 pages (approximately 100,000 characters) are filtered out during pre-processing. Resource management operates at multiple stages: the bloom filter component employs configurable size constraints, the model weighting stage can be limited to the top X candidates ranked by n-gram overlap frequency, and the clustering phase offers controls to process a specified subset of the 1M papers. These layered controls work together to ensure the system scales effectively while preserving detection accuracy and maintaining efficient resource utilization. All parameters and thresholds can be modified through pipeline configuration commands documented in the README.md file.

6 Pipeline Validation Strategy - How Would We Know It Works?

Evaluating our inference pipeline's effectiveness is challenging given the absence of ground-truth plagiarism labels in the arXiv corpus. We validate system performance through three key observations. First, the clustering process successfully groups papers from similar research fields, as evidenced by examining sample abstracts within clusters that consistently share topical coherence. Second, Bloom filter candidates demonstrate actual textual overlaps when manually inspected, confirming that high signal strength corresponds to genuine n-gram matches rather than false positives. Finally, the Siamese BERT model produces intuitive similarity scores-contextually different sentence pairs receive appropriately low scores while semantically similar passages score high, aligning with human judgment. These validation measures collectively demonstrate that each pipeline component performs as intended, providing confidence that the system effectively identifies paper pairs warranting further investigation for potential plagiarism.

7 Results & Conclusions

While our theoretical complexity analysis assumed a throughput of approximately 10,240 pairs per second (512 pairs per 0.05-second batch), the actual inference pipeline running on an NVIDIA H200 GPU with 16 CPUs and 64GB RAM achieved ~ 350 pairs per second, representing a $\sim 30\times$ discrepancy from our initial estimates. This performance gap highlights the difference between theoretical batching assumptions and real-world BERT inference overhead even on state-of-the-art hardware. Such degradation in inference time can be attributed to computational overhead components which we did not include in our initial complexity analysis, including tokenization (5.1% of total time) and post-processing (47% of total time) that were not accounted for in our simplified timing model. Therefore, we decided to limit our analysis to only the top candidates to make the task computationally feasible within our time constraints.

During inference, we observed numerous cases where the model assigned low similarity scores to text pairs that were nearly identical, indicating potential plagiarism. This issue arises from our training dataset's lack of near-plagiarism examples — specifically, cases where identical text is combined with non-identical additions or modifications. Consequently, when the model encounters such pairs during inference, it fails to recognize the high degree of similarity and produces inappropriately low similarity scores despite the substantial textual overlap.

Given this performance issue and the difficulty of obtaining training data with near-plagiarism examples, **we switched to using raw BERT embeddings for similarity computation.** Instead of our trained Siamese model, we employed the pre-trained BERT encoder to generate text representations and

calculated similarity using cosine distance between the embeddings, avoiding the limitations imposed by our training data distribution. Following the transition to the raw (pre-trained) BERT encoder, we observed genuine improvements in near-plagiarism detection performance. For example, when analyzing the pair `[1608.03858v1, 1611.06124v1]`, which contained 1156 actual n-gram overlaps and was subsequently processed by the BERT encoder, our initial model (trained on the PAN dataset) assigned a similarity score of 0.6 to the following chunk pair (Figure 10), whereas the pre-trained BERT model yielded a substantially higher similarity score of 0.94.

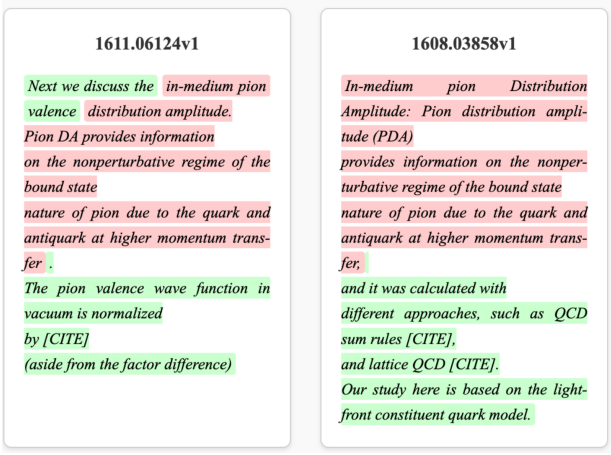

Figure 10: *Example of self-plagiarism that was detected*

This demonstrates that near-plagiarism cases of this nature are significantly more detectable using raw BERT compared to our PAN-trained Siamese model, since such instance types are absent from our training dataset*.

*It's worth noting that this paper pair shares the same authors and should have been excluded during the BERT encoder phase, but this filtering failed due to inadequate regex matching - in one paper, the authors' institutional affiliations were incorporated into their names, preventing the filter from recognizing them as identical authors.

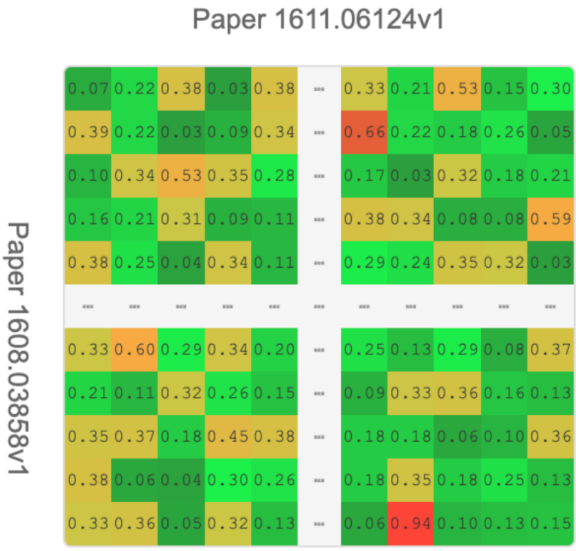

Figure 11: *A visual illustration of chunks comparison for the pair in example - 38 rows (number of chunks in paper 1608.03858v1) and 53 columns (number of chunks in paper 1611.06124v1) results in total of 2014 chunk comparisons*

For this reason, we assumed that paper pairs with several hundred actual overlapping n-grams are highly likely to originate from the same set of authors (which were not filtered out at this stage due to variations in author names). Such cases were therefore excluded from this phase of the analysis, allowing us to focus on medium-overlap instances instead. For example, when analyzing the pair `[1602.07674v1, 1611.04471v1]`, which contained 10 actual n-gram overlaps, our model assigned a high similarity score of 0.92 to the corresponding chunk pair (as shown in Figure 12). At first glance, this would appear to represent a case of near-plagiarism—potentially leading an automated system to flag Albash and Lidar for copying from Farhi and Harrow. However, upon closer inspection, a citation was found that explicitly credits the original authors, confirming that this instance does not constitute plagiarism.

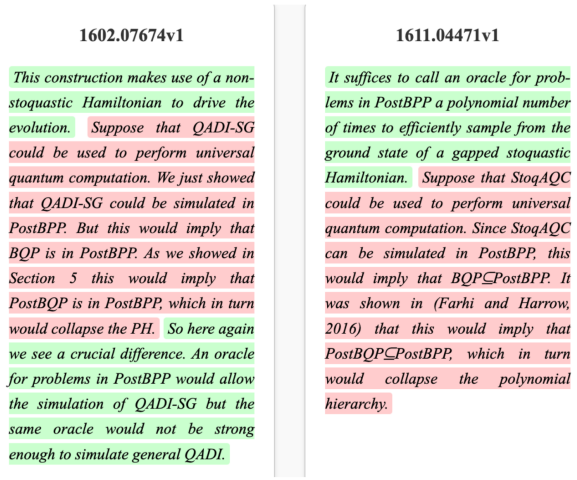

Figure 12: *Example of false plagiarism due to citation*

These two examples highlight the critical importance of the final pipeline stage manual review by a human since allegations of plagiarism are a serious matter in academic circles, and the current model does not yet have the capability to reliably determine whether plagiarism is present.

8 Limitations & Future Work

Although the proposed framework shows promising results, several limitations and future directions remain. Expanding the dataset to include more sophisticated obfuscation, such as translation-based plagiarism, could help the model capture complex transformations beyond paraphrasing and structural modifications. We fixed the n-gram length at 8, following prior research (Puflović & Stoimenov, 2015)[2], but did not explore alternatives; future hyperparameter tuning may improve performance. Developing a user-facing tool is another direction, allowing researchers to input passages or papers and receive candidate plagiarism instances - time complexity could be mitigated by assigning inputs to semantically relevant clusters before detailed comparisons, as in our pipeline. Improving detection of localized plagiarism - where plagiarized fragments appear within original text (e.g "... this section discusses prior work ... [plagiarized sentence] ... followed by new contributions ...") is also needed as such cases are absent from the PAN dataset, therefore the model fails to detect such cases, unless switched to raw BERT. Related to this is the difficulty of distinguishing legitimate citations from partial plagiarism. Authors may cite certain sections while still copying other parts without attribution. Finally, inference efficiency could be enhanced through model compression, such as quantization (reducing precision from float32 to int8) and knowledge distillation where a smaller "student" model is trained to mimic the outputs of a larger "teacher" model (Hinton, Vinyals & Dean, 2015)[3].

Code Availability & Implementation

Our full plagiarism detection pipeline is available on GitHub at https://github.com/Yaronot/PlagiarismDetection, including training scripts, inference code, and documentation for reproducing results. Please note that the current implementation is configured for the HUJI cluster, with specific GPU settings and paths to the arXiv dataset on that cluster. Users outside HUJI will need to adapt the code for their own computing environment and obtain independent access to the arXiv dataset.

References

1. Potthast, M.; Stein, B.; Eiselt, A.; Barrón-Cedeño, A.; Rosso, P. (2011). PAN Plagiarism Corpus 2011 (PAN-PC-11). Retrieved from https://zenodo.org/records/3250095

2. Puflović, D.; Stoimenov, L. V. (2015, September). Plagiarism detection in homework assignments and term papers. In *Proceedings of the Sixth International Conference on e-Learning (eLearning-2015)*, pages 99–105. Belgrade, Serbia: University of Niš.

3. Hinton, G.; Vinyals, O.; Dean, J. (2015). Distilling the Knowledge in a Neural Network. *arXiv:1503.02531*.